papergold
Date: / /

Day 25, 14/11/26

704 - Binary Search

Approach:- Binary search
1. Find the middle element. 2) If nums[mid] == target → return mid
3. If nums[mid] < target → search the right half.
4. If nums[mid] > target → search the left half.
5. Repeat until left > right.

Code

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return -1;
    }
}
```

Time Complexity : O(log n)
Space Complexity : O(1)

Description | Accepted | Editorial | Solutions | Submissions

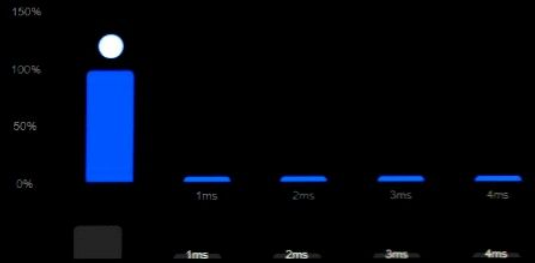
All Submissions

Runtime

0 ms | Beats **100.00%**

Memory

47.60 MB | Beats 96.11%



Code Java

```java
class Solution {
    public int search(int[] nums, int target) {

        int left = 0, right = nums.length - 1;
        while(left <= right){
```

Code

Java | Auto

```java
class Solution {
    public int search(int[] nums, int target) {

        int left = 0, right = nums.length - 1;
        while(left <= right){

            int mid = left +(right - left)/2;
            if(nums[mid] == target){
                return mid;

            }else if(nums[mid] > target){
                right = mid - 1;
            }else{
                left = mid + 1;
            }
        }
        return -1;
```

Saved

Ln 4, Col 47

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =

[-1,0,3,5,9,12]